UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 03

NOMBRE COMPLETO: Pérez Uribe José Alberto

Nº de Cuenta: 318143213

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 07

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 08-03-2026

CALIFICACIÓN: _____

Ejercicio Modelado Geométrico y Cámara Sintética:

1.- Generar una pirámide rubik (pyraminx) de 9 pirámides por cara. Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias.

El código ejemplo proporcionado en la práctica permite crear y renderizar primitivas geométricas independientes como cubos, cilindros, conos y pirámides, almacenándolas en meshList para después dibujarlas en la escena.

Para implementar el rompecabezas tipo Pyraminx, fue necesario modificar este enfoque. En lugar de utilizar varias primitivas genéricas, se diseñó un sistema que genera dinámicamente las piezas que conforman el rompecabezas.

Primero se definen los colores que se usarán para representar cada cara del Pyraminx. También se define un color negro para la base, que funciona como borde visual entre las subdivisiones triangulares y hace que cada pieza se distinga mejor.

```
const glm::vec3 COLOR_BLACK(0.0f, 0.0f, 0.0f);
const glm::vec3 COLOR_BLUE(0.10f, 0.40f, 0.95f);
const glm::vec3 COLOR_PINK(0.95f, 0.30f, 0.65f);
const glm::vec3 COLOR_GREEN(0.15f, 0.80f, 0.35f);
const glm::vec3 COLOR_YELLOW(0.98f, 0.85f, 0.15f);
```

En el código base, las figuras se crean de forma directa, por ejemplo:

```
CrearCubo();
CrearPiramideTriangular();
CrearCilindro(5, 1.0f);
CrearCono(25, 2.0f);
CrearPiramideCuadrangular();
```

En la implementación realizada, estas primitivas fueron sustituidas por una función especializada que construye todas las partes del Pyraminx.

```
const int ORDER = 3;
const float INSET = 0.2f;
CreatePyraminxMeshes(ORDER, INSET);
```

El Pyraminx está basado en un **tetraedro regular**, por lo que primero fue necesario crear una función que construyera esta geometría base, aquí se definieron cuatro vértices equidistantes normalizados para garantizar simetría entre las caras.

```
static Mesh* buildBaseTetra(const glm::vec3 v[4])
{
    vector<GLfloat> V = {
        v[0].x, v[0].y, v[0].z,
        v[1].x, v[1].y, v[1].z,
        v[2].x, v[2].y, v[2].z,
        v[3].x, v[3].y, v[3].z
    };

    vector<unsigned int> I = {
        0,1,2,
        0,3,1,
        0,2,3,
        1,3,2
    };

    Mesh* m = new Mesh();
    m->CreateMesh(V.data(), I.data(),
        static_cast<unsigned int>(V.size()),
        static_cast<unsigned int>(I.size()));
    return m;
}
```

Esta malla funciona como el soporte geométrico del rompecabezas.

Para dividir cada cara del tetraedro en piezas más pequeñas, se implementó un sistema de coordenadas baricéntricas discretas.

Esto permite calcular posiciones internas dentro del triángulo definido por tres vértices.

```
static glm::vec3 gridPoint(const glm::vec3& A, const glm::vec3& B, const glm::vec3& C,
    int i, int j, int N)
{
    float a = (float)i / (float)N;
    float b = (float)j / (float)N;
    float c = 1.0f - a - b;

    return c * A + a * B + b * C;
}
```

Este método permite generar una malla triangular regular dentro de cada cara.

Una vez definidos los puntos de la subdivisión, se construyen los triángulos que representan las piezas del Pyraminx.

Para lograr un efecto visual más claro entre piezas, cada triángulo se inserta ligeramente hacia adentro con respecto al centroide del triángulo original. Esto permite que la base negra del tetraedro sea visible como borde entre piezas.

```
static void pushTriInset(vector<GLfloat>& V, vector<unsigned int>& I,
    const glm::vec3& a0, const glm::vec3& b0, const glm::vec3& c0,
    float inset)
{
    glm::vec3 centroid = (a0 + b0 + c0) / 3.0f;

    glm::vec3 a = glm::mix(a0, centroid, inset);
    glm::vec3 b = glm::mix(b0, centroid, inset);
    glm::vec3 c = glm::mix(c0, centroid, inset);

    unsigned int base = static_cast<unsigned int>(V.size() / 3);

    V.push_back(a.x); V.push_back(a.y); V.push_back(a.z);
    V.push_back(b.x); V.push_back(b.y); V.push_back(b.z);
    V.push_back(c.x); V.push_back(c.y); V.push_back(c.z);

    I.push_back(base);
    I.push_back(base + 1);
    I.push_back(base + 2);
}
```

Este procedimiento genera la apariencia de piezas separadas característica del rubik.

Posteriormente se implementó una función que genera toda una cara triangular subdividida en múltiples piezas.

Esta función recorre una malla triangular y crea dos tipos de triángulos:

- triángulos normales
- triángulos invertidos

Esto permite llenar completamente la superficie triangular.

```
static Mesh* buildSubdividedFace(const glm::vec3& A, const glm::vec3& B, const glm::vec3& C,
    int N, float inset)
{
    vector<GLfloat> V;
    vector<unsigned int> I;

    for (int i = 0; i < N; ++i)
    {
        for (int j = 0; j < N - i; ++j)
        {
            glm::vec3 p0 = gridPoint(A, B, C, i, j, N);
            glm::vec3 p1 = gridPoint(A, B, C, i + 1, j, N);
            glm::vec3 p2 = gridPoint(A, B, C, i, j + 1, N);
            pushTriInset(V, I, p0, p1, p2, inset);

            if (i + j < N - 1)
            {
                glm::vec3 p3 = gridPoint(A, B, C, i + 1, j, N);
                glm::vec3 p4 = gridPoint(A, B, C, i + 1, j + 1, N);
                glm::vec3 p5 = gridPoint(A, B, C, i, j + 1, N);
                pushTriInset(V, I, p3, p4, p5, inset);
            }
        }
    }

    Mesh* m = new Mesh();
    m->CreateMesh(V.data(), I.data(),
        static_cast<unsigned int>(V.size()),
        static_cast<unsigned int>(I.size()));
    return m;
}
```

Finalmente se creó una función que reúne todas las partes: primero se genera la base tetraédrica y posteriormente las cuatro caras subdivididas.

```
static void CreatePyraminxMeshes(int N, float inset)
{
    meshList.clear();

    glm::vec3 v[4] = {
        glm::normalize(glm::vec3(1.0f,  1.0f,  1.0f)),
        glm::normalize(glm::vec3(-1.0f,  1.0f, -1.0f)),
        glm::normalize(glm::vec3(-1.0f, -1.0f,  1.0f)),
        glm::normalize(glm::vec3(1.0f, -1.0f, -1.0f))
    };

    meshList.push_back(buildBaseTetra(v));

    meshList.push_back(buildSubdividedFace(v[0], v[1], v[2], N, inset));
    meshList.push_back(buildSubdividedFace(v[0], v[3], v[1], N, inset));
    meshList.push_back(buildSubdividedFace(v[0], v[2], v[3], N, inset));
    meshList.push_back(buildSubdividedFace(v[1], v[3], v[2], N, inset));
}
```

De esta forma meshList contiene:

- base del tetraedro
- cuatro caras del rompecabezas

- El modelo se posiciona en la escena utilizando transformaciones de traslación, rotación y escalado.
- Esto permite visualizar el objeto correctamente en la cámara y observarlo desde distintos ángulos.

```
glm::mat4 baseModel(1.0f);
baseModel = glm::translate(baseModel, glm::vec3(0.0f, 0.0f, -2.0f));

baseModel = glm::rotate(baseModel, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
baseModel = glm::rotate(baseModel, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
baseModel = glm::rotate(baseModel, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));

baseModel = glm::rotate(baseModel, now * 0.6f, glm::vec3(0.0f, 1.0f, 0.0f));

baseModel = glm::scale(baseModel, glm::vec3(1.8f));
```

Primero se renderiza la base negra ligeramente más pequeña, lo que permite que sobresalgan los triángulos de color.

```
model = glm::scale(baseModel, glm::vec3(0.985f));
color = COLOR_BLACK;
meshList[0]->RenderMesh();
```

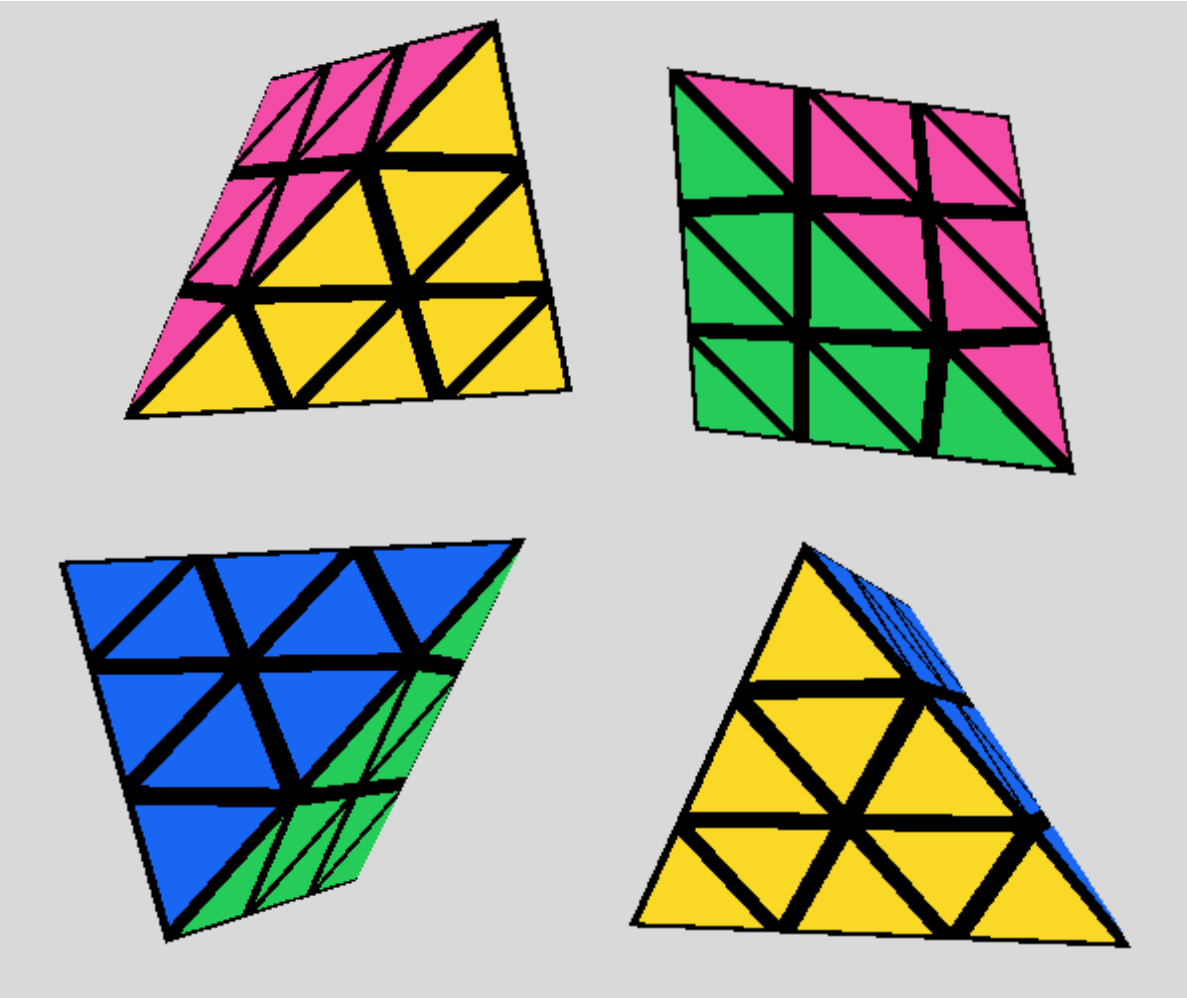
Posteriormente se dibujan las cuatro caras subdivididas con colores distintos.

```
const glm::vec3 faceColors[4] = {
    COLOR_BLUE,
    COLOR_PINK,
    COLOR_GREEN,
    COLOR_YELLOW
};

for (int f = 1; f <= 4; ++f)
{
    meshList[f]->RenderMesh();
}
```

La función main inicializa la ventana, los *shaders* y la cámara para poder visualizar la escena 3D. Posteriormente se genera el modelo del Pyraminx mediante la función CreatePyraminxMeshes.

Dentro del ciclo principal del programa se procesan las entradas del usuario, se actualizan las matrices de proyección, vista y modelo, y finalmente se renderizan las mallas que conforman el rubik almacenadas en meshList, aplicando las transformaciones necesarias para visualizarlo correctamente en la escena.



El modelo cuenta con una rotación continua alrededor del eje **Y**, lo que permite observar la pirámide desde distintos ángulos. Adicionalmente, se puede controlar la cámara mediante el teclado y el mouse, lo que permite desplazarse por la escena y cambiar el punto de vista del objeto.

2.- Problemas encontrados y cómo se resolvieron

Uno de los primeros problemas fue que al inicio el modelo no se generaba correctamente. Intenté construir el Pyraminx directamente dentro del main, pero el código se volvió difícil de manejar y la geometría no se formaba como esperaba. Para solucionarlo decidí separar la lógica en funciones específicas que se encargaran de generar cada parte del modelo, como la base tetraédrica y las caras subdivididas.

Otro problema que encontré fue al momento de generar las subdivisiones de cada cara. En algunos intentos los triángulos no coincidían correctamente y aparecían espacios o piezas mal acomodadas. Esto se resolvió implementando una función que calcula puntos dentro del triángulo utilizando coordenadas, lo que permitió generar una malla triangular más ordenada.

En general, los problemas estuvieron más relacionados con la organización del código y la generación correcta de la geometría que con errores de sintaxis. Una vez separada la lógica en funciones y ajustados los cálculos de las subdivisiones, el modelo comenzó a renderizarse correctamente.

3.- Conclusión

a) Los ejercicios del reporte: Complejidad y explicación

La práctica tuvo una dificultad alta. El código base ya tenía muchas partes listas como la creación de primitivas y el manejo de la cámara, pero fue necesario modificarlo para poder construir el Pyraminx. La parte que tomó más tiempo fue entender cómo generar las caras triangulares y cómo subdividirlas para que se vieran como las piezas del rompecabezas.

También fue importante entender cómo funcionan las transformaciones en OpenGL, ya que con las matrices de modelo, vista y proyección se controla la posición, orientación y tamaño del objeto dentro de la escena.

b) Comentarios generales

En general la práctica fue clara respecto a lo que se pedía, aunque al inicio puede costar un poco entender cómo se generan las figuras a partir de triángulos y cómo se acomodan los vértices en el espacio. También ayuda bastante dividir el código en funciones para que la construcción del modelo sea más fácil de entender.

c) Conclusión

Esta práctica ayudó a entender mejor cómo se pueden construir modelos 3D a partir de geometría básica en OpenGL. También permitió practicar el uso de transformaciones y el manejo de la cámara para observar el objeto desde diferentes ángulos.

En general fue útil para comprender mejor cómo funciona la estructura de un programa gráfico y cómo modificar el código base para crear una figura más compleja.

Bibliografia

- Sieger, D. (2021, May 3). Generating primitive shapes in C++. Daniel Sieger.
- https://danielsieger.com/blog/2021/05/03/generating-primitive-shapes.html
- Ahn, S. H. (n.d.). OpenGL. https://www.songho.ca/opengl/index.html
- ConeGeometry – three.js docs. (n.d.).
- https://threejs.org/docs/#ConeGeometry
- Ahn, S. H. (n.d.). OpenGL cylinder, prism & pipe.
- https://www.songho.ca/opengl/gl_cylinder.html
- CylinderGeometry – three.js docs. (n.d.).
- https://threejs.org/docs/#CylinderGeometry
- Cylinder definition and properties - Math Open Reference. (n.d.).
- https://www.mathopenref.com/cylinder.html
- Ahn, S. H. (n.d.). OpenGL Sphere.
- https://www.songho.ca/opengl/gl_sphere.html
- LearnOpenGL - Scene Graph. (n.d.). https://learnopengl.com/Guest-
- Articles/2021/Scene/Scene-Graph
- SphereGeometry – three.js docs. (n.d.).
- https://threejs.org/docs/#SphereGeometry